

OPERATING SYSTEMS (UE25CS242A)

Unit 1 – CPU Scheduling Algorithms – MQ, MLFQ & Multiprocessor Scheduling

Multilevel Queue Scheduling

In **multilevel queue scheduling**, the **ready queue** is **partitioned into multiple separate queues**, based on the type of process.

Typical Queues

- **Foreground queue** → Interactive processes
- **Background queue** → Batch processes

Key Characteristics

- Each process is **permanently assigned** to one queue
- Each queue has its **own scheduling algorithm**
 - Foreground → Round Robin (RR)
 - Background → First-Come, First-Served (FCFS)

CPU Scheduling Between Queues

Scheduling must also be done **between the queues**, not just within a queue.

Methods

1. Fixed Priority Scheduling

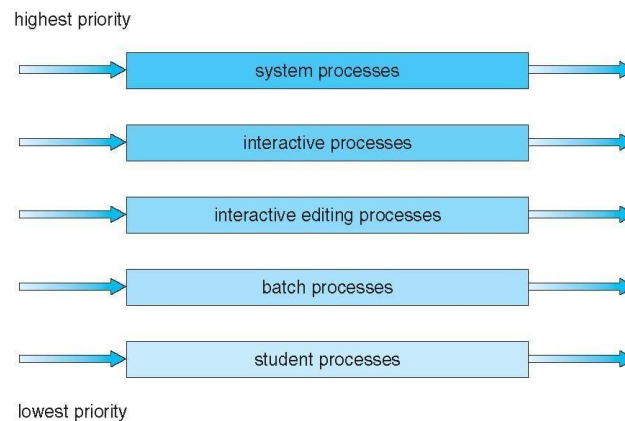
- Foreground queue has higher priority than background queue
- All foreground processes are executed before background processes
- Can lead to **starvation** of background processes

2. Time-Slice Scheduling Between Queues

- Each queue gets a certain percentage of CPU time
- Example:
 - 80% CPU time → Foreground (RR)
 - 20% CPU time → Background (FCFS)

Multilevel Queue Scheduling – Priority Behavior

- Each queue has **absolute priority** over lower-priority queues
- A process in a lower queue **cannot execute** unless all higher-priority queues are empty
- If a higher-priority process arrives:
 - The currently running lower-priority process is **preempted**



Multilevel Feedback Queue (MLFQ) Scheduling

Multilevel feedback queue scheduling allows **processes to move between queues**.

Key Idea

- Unlike multilevel queue scheduling:
 - Processes are **not permanently assigned** to a queue
- Supports **aging**
 - Prevents starvation by gradually moving processes to higher-priority queues

Parameters Defining an MLFQ Scheduler

An MLFQ scheduler is defined by:

1. Number of queues
2. Scheduling algorithm for each queue
3. Method used to **upgrade** a process
4. Method used to **demote** a process
5. Method used to determine **which queue a process enters** when it needs CPU service

Example of Multilevel Feedback Queue

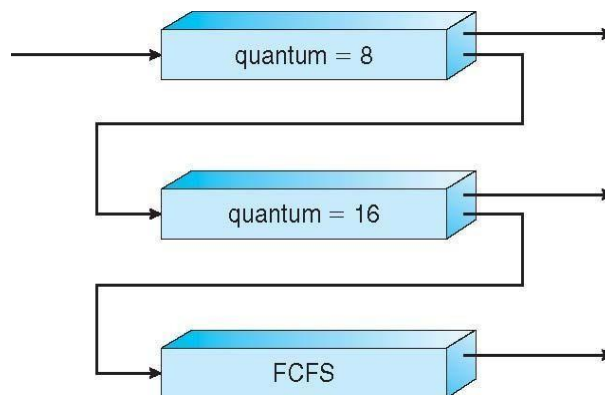
Consider **three queues**:

- $Q_0 \rightarrow$ Round Robin, time quantum = **8 ms**
- $Q_1 \rightarrow$ Round Robin, time quantum = **16 ms**
- $Q_2 \rightarrow$ FCFS

Execution Rules

1. A new process enters Q_0
2. It receives **8 ms**
3. If not completed:
 - Moved to Q_1

4. In Q_1 :
 - Receives **16 ms**
5. If still not completed:
 - Moved to Q_2 (FCFS)



MLFQ – Queue Priority Rules

- Scheduler always executes processes in Q_0 first
- Q_1 is considered only when Q_0 is empty
- Q_2 is considered only when both Q_0 and Q_1 are empty

Preemption Rules

- Process arriving in Q_0 preempts processes in Q_1 and Q_2
- Process arriving in Q_1 preempts processes in Q_2

This ensures responsiveness for interactive processes.

Multiple-Processor Scheduling

When multiple CPUs are available:

- **Load sharing** becomes possible
- Scheduling becomes more complex

Asymmetric Multiprocessing (AMP)

Characteristics

- One processor (master) handles:
 - Scheduling decisions
 - I/O processing
 - System activities
- Other processors execute only user code

Advantages

- Simple design
- Only one processor accesses system data structures

- Less synchronization overhead

Symmetric Multiprocessing (SMP)

Characteristics

- Each processor is **self-scheduling**
- Processes may be:
 - In a common ready queue, or
 - In separate per-processor queues
- All modern operating systems support **SMP**

Processor Affinity

Concept

- When a process runs on a processor:
 - Its data gets loaded into that processor's cache
- Migrating the process to another processor:
 - Invalidates old cache
 - Requires cache repopulation
- This is **expensive**

To reduce this cost, OS tries to keep a process on the same processor.

Types of Processor Affinity

1. Soft Affinity

- OS attempts to keep a process on the same processor
- Migration is still possible

2. Hard Affinity

- OS allows process to specify which processors it may run on
- Implemented using system calls

Load Balancing

In SMP systems, load must be evenly distributed.

Goal

- Ensure all processors are utilized effectively

Requirement

- Mainly needed when each processor has its **own ready queue**

Load Balancing Techniques

1. Push Migration

- A periodic task checks processor loads
- Moves processes from overloaded processors to idle or lightly loaded ones

2. Pull Migration

- An idle processor pulls a process from a busy processor

Push and pull migration can be used **together**.